
Requirements, Design and Tasks for rxnet

C Implementation — Reactive execution networks

2026-04-18

Contents

Requirements Document: rxnet — C Implementation	7
Introduction	7
Glossary	7
Requirements	8
Requirement 1: Shared Synchronous Phase Model	8
Requirement 2: Context and Input Snapshot Handling	8
Requirement 3: Deferred Action Execution	8
Requirement 4: Generic Node Runtime API	9
Requirement 5: FSM Semantics	9
Requirement 6: FSM Shared Input Access	10
Requirement 7: Petri Net Semantics	10
Requirement 8: PN Data Validation and Error Behavior	11
Requirement 9: Frontend Lifecycle and Memory Management	11
Requirement 10: Integration Patterns and Examples	11
Requirement 11: Reusable CLI FSM Utility (Example Layer)	12
Requirement 12: Node Embedding and Typed Polymorphism	12
Requirement 13: Init/Create API Pattern	13
Requirement 14: Execution Model Constraints	13
Requirement 15: Concurrency Integration Patterns	14
Design Document: rxnet — C Implementation	15
Overview	15
Architecture	16
High-Level Architecture	16
Component Architecture	16
Components and Interfaces	17
Core Runtime	17
Context	17
FSM Frontend	18
Petri Net Frontend	19
Host Integration Layer	20

Reusable CLI FSM Utility (Example Layer)	21
Data Models	22
Runtime Model	22
FSM Model	23
PN Model	23
Correctness Properties	24
Property 1: Phase Ordering Determinism	24
Property 2: Snapshot Consistency Within Tick	24
Property 3: Deferred Action Isolation	24
Property 4: Deferred Queue Reset	24
Property 4b: Deferred Queue Overflow Determinism	24
Property 5: Runtime Node Contract Safety	24
Property 6: Node Capacity Enforcement	25
Property 6b: Init Capacity Bound Checks	25
Property 7: FSM First-Match Transition Rule	25
Property 8: FSM No-Match Stability	25
Property 9: FSM Deferred Action Behavior	25
Property 10: FSM Input Snapshot Access	25
Property 11: PN Transition Enablement	25
Property 12: PN Guard Enforcement	26
Property 13: PN Commit Delta Correctness	26
Property 14: PN Arc Validation	26
Property 15: Lifecycle Idempotent Safety	26
Property 16: Context Ownership Boundary	26
Property 17: Example Executability	26
Property 18: Host-Controlled Scheduling	26
Error Handling	27
Error Categories	27
Error Handling Strategy	27
Testing Strategy	27
Unit and Integration Tests	27
Property-Based Testing Configuration	27
Code Quality and Development Standards	28
Non-Goals and Out-of-Scope	28
Implementation Plan: rxnet — C Implementation	29
Overview	29
Tasks	29

Notes	36
-----------------	----

List of Figures

1	High-level architecture of the <code>rxnet</code> C runtime and model frontends	16
---	---	----

Requirements Document: rxnet — C Implementation

Introduction

This document specifies the requirements for the C implementation of [rxnet](#), a reactive synchronous runtime library that supports two model families: Finite State Machines (FSM) and Petri Nets (PN). The C implementation provides a shared phase-based execution core and language frontend in C11.

The goal of this document is to capture what the C library provides today, so future changes can be validated and reflected here first.

Glossary

- **RXNET_System**: The complete [rxnet](#) C library surface
- **Core_Runtime**: Shared synchronous execution engine with phase-based ticking
- **Context**: Execution context containing live inputs, latched inputs, and deferred actions
- **Node**: Runtime unit with [evaluate](#) and [commit](#) phases
- **Deferred_Action**: Action scheduled during commit and executed after all commits
- **FSM_Runtime**: FSM frontend wrapper over the core runtime
- **FSM_Machine**: State machine node with transitions, guards, and optional actions
- **FSM_Transition**: Rule from [from_state](#) to [to_state](#), with optional guard/action
- **PN_Runtime**: Petri Net frontend wrapper over the core runtime
- **PN_Net**: Petri net node with places, transitions, and firing flags
- **PN_Transition**: Petri transition with consume/produce arcs, optional guard/action
- **Arc**: Petri arc with [place_id](#) and [weight](#)
- **Tick**: One complete synchronous cycle: latch -> evaluate -> commit -> deferred actions
- **Latched_Inputs**: Immutable snapshot of current inputs used during one tick
- **Config_Header**: Compile-time configuration header ([rxnet/config.h](#)) that defines fixed runtime capacities

Requirements

Requirement 1: Shared Synchronous Phase Model

User Story: As a model developer, I want deterministic phase-based execution, so that all nodes observe a consistent input snapshot and state updates happen predictably.

Acceptance Criteria

1. WHEN a tick starts, THE Core_Runtime SHALL latch input data before node evaluation
2. WHEN latching completes, THE Core_Runtime SHALL evaluate all registered nodes
3. WHEN evaluation completes, THE Core_Runtime SHALL commit all registered nodes
4. WHEN commit completes, THE Core_Runtime SHALL execute deferred actions and clear the queue
5. THE Tick order SHALL remain `latch -> evaluate -> commit -> deferred actions` in the C implementation

Requirement 2: Context and Input Snapshot Handling

User Story: As an integrator, I want each node to manage its own input snapshot, so that guards read a stable per-node snapshot during each tick.

Acceptance Criteria

1. THE Context SHALL NOT own any global input buffer; input ownership stays with each node's `user` payload or the host application
2. EACH node SHALL read and latch its own inputs inside its `latch_inputs` callback (phase 1 of the tick)
3. THE `latch_inputs` callback SHALL run before `evaluate`, guaranteeing that guards observe a stable snapshot for the duration of one tick
4. THE runtime SHALL NOT copy or free any application-owned input memory

Requirement 3: Deferred Action Execution

User Story: As a runtime user, I want actions to run after all commits, so that side effects are deferred and model state commits are consistent.

Acceptance Criteria

1. WHEN a node commits with a proposed action, THE Context SHALL enqueue the action for deferred execution
2. THE Core_Runtime SHALL run deferred actions only after all node commits in the tick
3. THE deferred action queue SHALL be cleared after execution
4. THE deferred-action queue capacity SHALL be fixed at initialization from compile-time configuration (`rxnet/config.h`)
5. WHEN enqueue parameters are invalid (`ctx` or function is null), THE API SHALL return an error code (-1)
6. WHEN deferred-action capacity is exhausted, THE enqueue API SHALL return -1 without allocating memory

Requirement 4: Generic Node Runtime API

User Story: As a library user, I want a generic runtime abstraction, so that different model families can share one execution engine.

Acceptance Criteria

1. THE Node contract SHALL include `evaluate` and `commit` callbacks
2. THE runtime SHALL support runtime initialization with explicit node capacity
3. WHEN adding nodes beyond configured capacity, THE runtime SHALL reject the operation with -1
4. WHEN `tick` is called with invalid runtime/context, THE API SHALL return -1
5. WHEN initializing runtime, THE requested node capacity SHALL be less than or equal to the compile-time configured maximum node capacity

Requirement 5: FSM Semantics

User Story: As an FSM author, I want first-match transition evaluation with deferred actions, so that machine behavior is deterministic and clear.

Acceptance Criteria

1. WHEN an FSM machine evaluates, THE machine SHALL reset `next_state` to current state and clear proposed action

2. THE FSM evaluator SHALL scan transitions in declaration order and select the first transition whose `from_state` matches and whose guard is true (or absent)
3. WHEN no transition matches, THE machine SHALL remain in the current state
4. WHEN committing, THE machine SHALL update current state to `next_state`
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action (not executed inline during evaluate/commit)

Requirement 6: FSM Shared Input Access

User Story: As an embedded integrator, I want multiple machines to share one latched input snapshot, so that guard evaluation is simple and deterministic.

Acceptance Criteria

1. THE runtime SHALL allow multiple FSM machines to share one context input struct
2. THE FSM guard/action callbacks SHALL receive the shared latched context and machine user payload
3. FSM guard evaluation SHALL proceed directly with transition guard checks

Requirement 7: Petri Net Semantics

User Story: As a Petri Net author, I want place/transition execution with guards and arc validation, so that token flow is modeled consistently.

Acceptance Criteria

1. WHEN a PN net evaluates, THE net SHALL copy `places` into `next_places` and reset transition fire flags
2. THE PN evaluator SHALL mark a transition as fireable only if consume/produce arcs are valid and consume tokens are available
3. WHEN a transition guard exists, THE transition SHALL fire only if the guard returns true
4. WHEN committing, THE net SHALL apply consume/produce deltas for each fire-flagged transition
5. WHEN a transition action exists, THE action SHALL be enqueued as a deferred action

Requirement 8: PN Data Validation and Error Behavior

User Story: As a developer, I want invalid Petri model data to be detected, so that configuration errors fail early.

Acceptance Criteria

1. WHEN initializing a PN net, THE API SHALL validate all consume/produce arc lists and reject invalid models with `-1`
2. THE PN arc validator SHALL reject arcs with out-of-range `place_id` or negative `weight`
3. WHEN PN net allocation fails, THE API SHALL free partial allocations and return `-1`

Requirement 9: Frontend Lifecycle and Memory Management

User Story: As a C integrator, I want explicit initialization and cleanup APIs, so that runtime memory is managed safely.

Acceptance Criteria

1. THE API SHALL provide runtime init/free functions for both FSM and PN frontends, and optional create/destroy convenience constructors
2. THE API SHALL provide model registration functions (`add_machine`, `add_net`) before ticking
3. THE API SHALL provide model-specific cleanup for PN net internal buffers (`rx_pn_net_free`)
4. WHEN freeing runtime/context, THE API SHALL reset internal pointers and counters to neutral values
5. WHEN null pointers are passed to free functions, THE functions SHALL return safely without crashing
6. FOR entities that expose `_create`, THE API SHALL expose matching `_destroy` and `_init` entry points

Requirement 10: Integration Patterns and Examples

User Story: As a user evaluating the library, I want runnable examples in C, so that I can understand expected integration patterns.

Acceptance Criteria

1. THE repository SHALL include runnable C examples for FSM and PN under `c/examples`
2. THE repository SHALL include an ESP-IDF FSM integration example with periodic ticking and ISR-driven input updates
3. THE host CLI FSM example (`c/examples/fsm/00-light/main_cli.c`) SHALL keep runtime wiring minimal (machine creation/registration + periodic tick loop), delegating command parsing to a dedicated CLI FSM

Requirement 11: Reusable CLI FSM Utility (Example Layer)

User Story: As an example integrator, I want a reusable CLI FSM utility with command registration and contextual user data, so that multiple examples can share the same terminal command loop behavior without duplicating logic.

Acceptance Criteria

1. THE example layer SHALL provide a reusable CLI FSM module independent from the `00-light` domain model
2. THE CLI FSM module SHALL support registering multiple commands by name via a public registration API
3. EACH registered command SHALL support a per-command `user_data` payload passed to the command callback
4. THE CLI FSM machine data SHALL expose a machine-level `user_data` pointer for integration-specific shared context
5. THE CLI FSM module SHALL encapsulate terminal raw mode entry and restoration

Requirement 12: Node Embedding and Typed Polymorphism

User Story: As a C integrator, I want model structs to embed a base node type, so that runtime polymorphism is type-structured and avoids untyped `void*` `self` adapters.

Acceptance Criteria

1. THE `rx_node` interface SHALL support polymorphism without a `void*` `self` field
2. THE FSM machine struct SHALL embed `rx_node` as its base member
3. THE PN net struct SHALL embed `rx_node` as its base member

4. THE runtime node registry SHALL store node pointers to embedded base nodes
5. THE tick loop SHALL invoke `evaluate/commit` through the embedded node base
6. THE frontend runtime wrappers (`rx_fsm_runtime`, `rx_pn_runtime`) SHALL embed `rx_runtime` as first member to allow safe upcast to base runtime pointer

Requirement 13: Init/Create API Pattern

User Story: As a C developer, I want both `_init` and `_create` APIs, so that I can choose deterministic preallocated memory or convenience allocation consistently across entities.

Acceptance Criteria

1. THE API SHALL keep `_init` functions for caller-owned memory initialization
2. THE API SHALL provide `_create` functions that allocate memory and delegate to `_init`
3. THE API SHALL provide matching `_destroy` functions for `_create` allocations
4. THE `_create` APIs SHALL perform allocation only during creation time and fail with null on allocation/init errors
5. THE `_destroy` APIs SHALL be null-safe

Requirement 14: Execution Model Constraints

User Story: As a platform engineer, I want clear operational constraints and concurrency patterns, so that I can integrate `rxnet` correctly in bare-metal, RTOS, and threaded environments.

Acceptance Criteria

1. THE runtime SHALL be single-threaded per runtime instance unless external synchronization is provided by the integrator; `rx_tick` is not re-entrant
2. THE library SHALL not include built-in networking, persistence, or REST APIs
3. THE library SHALL leave scheduling policy (tick frequency/loop ownership) to the host application
4. THE library SHALL leave I/O side effects to user-defined action callbacks
5. THE library SHALL support using one shared typed input structure across multiple model nodes in the same runtime context
6. THE tick path (`latch`, `evaluate`, `commit`, `run deferred actions`) SHALL perform no heap allocation or reallocation

7. THE fixed runtime capacities used by the tick path SHALL be configurable through `rxnet/config.h`

Requirement 15: Concurrency Integration Patterns

User Story: As an embedded or server integrator, I want documented patterns for using `rxnet` with cyclic executives, RTOS tasks, and OS threads, so that I can integrate without guessing.

Acceptance Criteria

1. **Cyclic executive (bare-metal / single context):** Calling `rx_tick` in a periodic loop with inputs updated from ISRs using word-sized atomic variables SHALL be safe without additional locking. Node `latch_inputs` callbacks take the snapshot at the start of each tick.
2. **OS threads (POSIX / Win32):** An integrator SHALL be able to run `rx_tick` from a dedicated tick thread and update inputs from other threads by protecting both the input write and the tick call with a single mutex:

```
1 /* writer thread / ISR: */
2 pthread_mutex_lock(&lock);
3 inputs.button = 1;
4 pthread_mutex_unlock(&lock);
5
6 /* tick thread: */
7 pthread_mutex_lock(&lock);
8 rx_fsm_tick(&runtime);
9 pthread_mutex_unlock(&lock);
```

3. **RTOS cooperative tasks (FreeRTOS / Zephyr):** An integrator SHALL be able to wake the tick task from an ISR or driver task using a task notification or semaphore, then call `rx_tick` inside the tick task. No additional lock is required if all input writes happen in the same task context or from ISRs using atomic word writes.
4. THE library SHALL make no assumption about which execution context calls `rx_tick`; the integrator owns that decision entirely.

Design Document: rxnet — C Implementation

Overview

This document outlines the design for the C implementation of `rxnet`, a synchronous reactive runtime library with two model frontends (Finite State Machine and Petri Net) built on a shared phase-based core.

The core design decision is to centralize tick orchestration (`latch` → `evaluate` → `commit` → `deferred actions`) and let each model frontend implement only model-specific behavior on top of a generic node contract.

The design emphasizes deterministic execution, explicit input snapshotting, deferred side effects, zero heap allocation during the tick path, and lightweight integration into host loops (CLI, embedded, or application-managed schedulers).

Architecture

High-Level Architecture

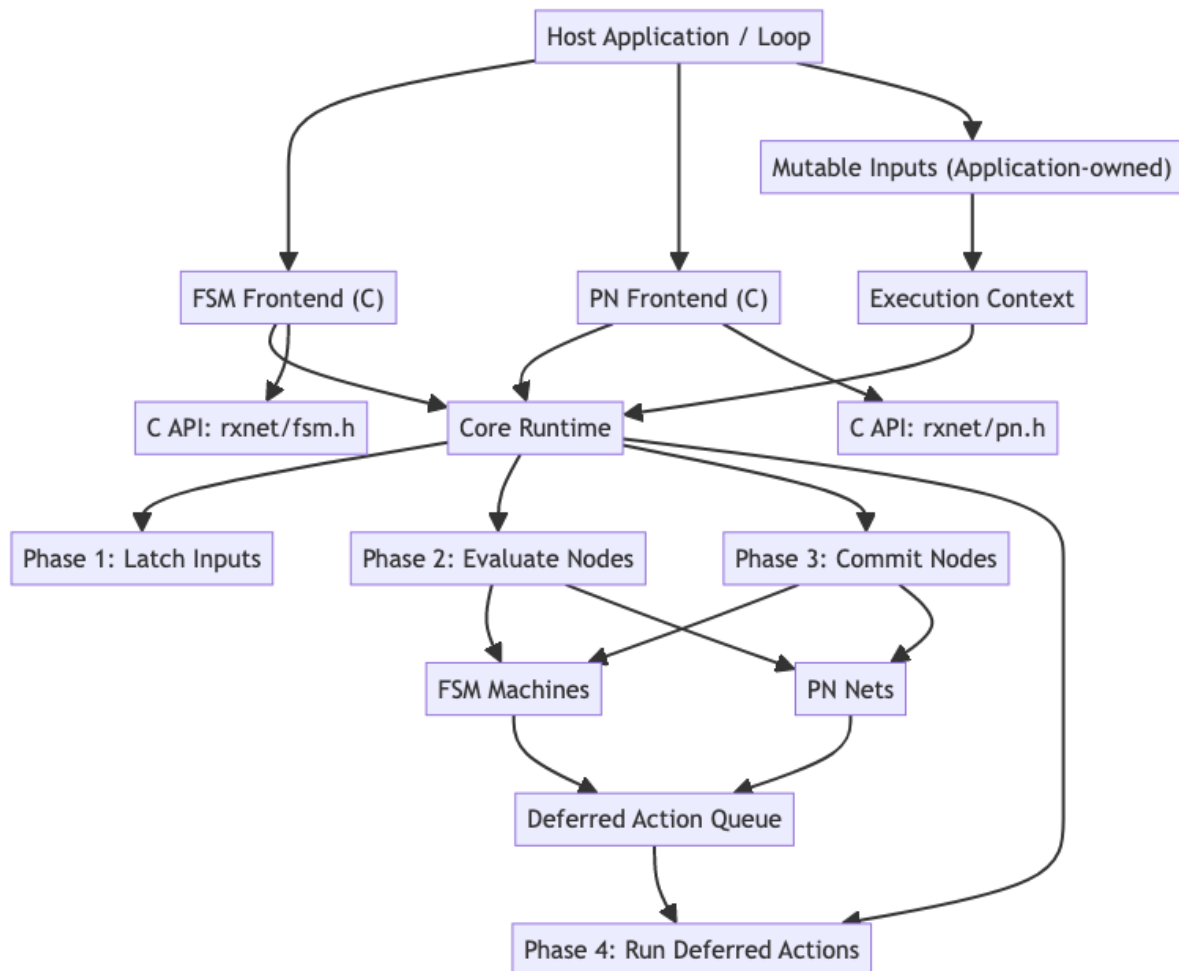


Figure 1: High-level architecture of the `rxnet` C runtime and model frontends

Component Architecture

The system follows a layered runtime architecture:

1. **Host Integration Layer:** Application-owned inputs, tick scheduling, and output side effects
2. **Model Frontends:** FSM and PN model APIs
3. **Core Runtime Layer:** Generic node container and phase-ordered tick execution
4. **Execution Context Layer:** Live inputs, latched snapshot, deferred action queue

5. **Model Nodes:** Concrete node implementations (`rx_fsm_machine`, `rx_pn_net`) with evaluate/commit logic

Components and Interfaces

Core Runtime

Responsibilities: - Maintain a list of executable nodes - Execute deterministic tick phases - Coordinate context latch and deferred actions - Provide minimal lifecycle surface (`init/add/tick/free`)

Key Interfaces:

```

1  typedef struct rx_node rx_node;
2
3  typedef struct rx_node_vtable {
4      void (*evaluate)(rx_node *node, rx_context *ctx);
5      void (*commit)(rx_node *node, rx_context *ctx);
6  } rx_node_vtable;
7
8  typedef struct rx_node {
9      const rx_node_vtable *vtable;
10 } rx_node;
11
12 int rx_runtime_init(rx_runtime *rt, rx_context *ctx, size_t
    node_capacity);
13 int rx_runtime_add_node(rx_runtime *rt, rx_node *node);
14 int rx_tick(rx_runtime *rt);
15 void rx_runtime_free(rx_runtime *rt);

```

Context

Responsibilities: - Queue and execute deferred actions after all commits - Serve as the shared argument passed to every node phase callback

Design Notes: - The context holds **no global input buffer**. Each node manages its own inputs (typically inside its `user` payload). The `latch_inputs` callback (phase 1 of the tick) is where each node takes its own stable snapshot before evaluation. - Deferred-action queue uses a fixed-size array configured in `rxnet/config.h`; enqueue returns `-1` on overflow with no `malloc/realloc`

Key Interfaces:

```

1  int rx_context_init(rx_context *ctx);
2  rx_context *rx_context_create(void);
3  void rx_context_free(rx_context *ctx);

```

```

4 void rx_context_destroy(rx_context *ctx);
5 int  rx_context_enqueue_deferred_action(rx_context *ctx,
    rx_deferred_action_fn fn, void *user);
6 void rx_context_run_deferred_actions(rx_context *ctx);

```

FSM Frontend

Responsibilities: - Define machine transitions (*from_state*, *to_state*, *guard*, *action*) - Evaluate first valid transition in declaration order - Commit next state and enqueue optional deferred action - Read shared latched inputs from context in guards/actions

Key Interfaces:

```

1 typedef struct rx_fsm_runtime {
2     rx_runtime runtime; /* base runtime (first member) */
3     rx_context context;
4 } rx_fsm_runtime;
5
6 typedef struct rx_fsm_machine {
7     rx_node node; /* base node */
8     /* ... */
9 } rx_fsm_machine;
10
11 typedef int (*rx_fsm_guard_fn)(const rx_fsm_context *ctx, void *user);
12 typedef void (*rx_fsm_action_fn)(rx_fsm_context *ctx, void *user);
13
14 void rx_fsm_machine_init(
15     rx_fsm_machine *machine,
16     const char *name,
17     int initial_state,
18     const rx_fsm_transition *transitions,
19     size_t transition_count,
20     void *user,
21     rx_fsm_node_phase_fn latch_inputs, /* void (*)(rx_fsm_context*,
    void* user) */
22     rx_fsm_node_phase_fn dump_outputs /* void (*)(rx_fsm_context*,
    void* user) */
23 );
24 int rx_fsm_runtime_add_machine(rx_fsm_runtime *runtime, rx_fsm_machine
    *machine);
25 int rx_fsm_tick(rx_fsm_runtime *runtime);
26
27 rx_fsm_machine *rx_fsm_machine_create(/* same args as init */);
28 void rx_fsm_machine_destroy(rx_fsm_machine *machine);

```

Petri Net Frontend

Responsibilities: - Represent places and transitions with consume/produce arcs - Evaluate transition enablement by token availability and guards - Apply transition deltas in commit phase - Enqueue transition actions as deferred side effects

Key Interfaces:

```

1  typedef struct rx_pn_runtime {
2      rx_runtime runtime; /* base runtime (first member) */
3      rx_context context;
4  } rx_pn_runtime;
5
6  typedef struct rx_pn_net {
7      rx_node node; /* base node */
8      /* ... */
9  } rx_pn_net;
10
11 typedef struct rx_pn_arc {
12     size_t place_id;
13     int weight;
14 } rx_pn_arc;
15
16 typedef struct rx_pn_transition {
17     const rx_pn_arc *consume;
18     size_t consume_count;
19     const rx_pn_arc *produce;
20     size_t produce_count;
21     rx_pn_guard_fn guard;
22     rx_pn_action_fn action;
23 } rx_pn_transition;
24
25 int rx_pn_net_init(
26     rx_pn_net *net,
27     const char *name,
28     const int *initial_places,
29     size_t place_count,
30     const rx_pn_transition *transitions,
31     size_t transition_count,
32     void *user,
33     rx_pn_node_phase_fn latch_inputs, /* void (*)(rx_pn_context*,
34                                         void* user); NULL → noop */
35     rx_pn_node_phase_fn dump_outputs /* void (*)(rx_pn_context*,
36                                         void* user); NULL → noop */
37 );
38
39 rx_pn_net *rx_pn_net_create(/* same args as init */);
40 void rx_pn_net_destroy(rx_pn_net *net);

```

Host Integration Layer

Responsibilities: - Own and mutate live inputs before each tick (via node `user` payloads) - Invoke tick in a periodic or event-driven loop - Reset edge-triggered inputs when required by domain logic - Implement side effects in action callbacks

Concurrency patterns — all supported without library changes:

Pattern	Who calls <code>rx_tick</code>	Input writes	Synchronization needed
Cyclic executive	Single loop / main task	ISR (word-sized atomic)	None — latch takes snapshot at tick start
OS threads	Dedicated tick thread	Other threads	Mutex around input write AND tick call
RTOS cooperative	Tick task (woken by notification)	ISR or driver task (atomic)	None for atomic writes; semaphore to wake tick task

Cyclic executive (bare-metal):

```

1  /* ISR: word-sized write is atomic on ARM Cortex-M */
2  void BUTTON_IRQHandler(void) { inputs.button = 1; }
3
4  /* Main loop: */
5  while (1) {
6      rx_fsm_tick(&runtime);
7      /* dump outputs, sleep to next period */
8  }
```

Threads (POSIX):

```

1  /* Writer thread: */
2  pthread_mutex_lock(&lock);
3  inputs.button = 1;
4  pthread_mutex_unlock(&lock);
5
6  /* Tick thread: */
7  while (1) {
8      pthread_mutex_lock(&lock);
9      rx_fsm_tick(&runtime);
10     pthread_mutex_unlock(&lock);
11     nanosleep(&period, NULL);
12 }
```

RTOS cooperative (FreeRTOS):

```

1  /* ISR or driver task: */
2  void button_isr(void) {
3      inputs.button = 1;          /* atomic word write */
4      xTaskNotifyGive(tick_task_handle); /* wake tick task */
5  }
6
7  /* Tick task: */
8  void tick_task(void *arg) {
9      while (1) {
10         ulTaskNotifyTake(pdTRUE, portMAX_DELAY);
11         rx_fsm_tick(&runtime);
12     }
13 }

```

Mixing FSM and PN in a single base runtime:

`rx_fsm_machine` and `rx_pn_net` both embed `rx_node` as their first member. Any number of them can be registered with one `rx_runtime` and advanced by a single `rx_tick` call:

```

1  rx_context      ctx;
2  rx_runtime      rt;
3  rx_fsm_machine light_a; /* FSM node */
4  rx_pn_net       light_b; /* PN node */
5
6  rx_context_init(&ctx);
7  rx_runtime_init(&rt, &ctx, 2);
8
9  light_fsm_create(&light_a, BUTTON_A, LIGHT_A);
10 light_pn_init(&light_b, BUTTON_B, LIGHT_B);
11
12 rx_runtime_add_node(&rt, &light_a.node);
13 rx_runtime_add_node(&rt, &light_b.node);
14
15 while (1) {
16     rx_tick(&rt);          /* advances both light_a (FSM) and light_b (
17                             PN) */
17 }

```

See `examples/mixed/main_cli.c` for a full working example.

Reusable CLI FSM Utility (Example Layer)

Responsibilities: - Run non-blocking character intake from `stdin` inside an FSM machine - Buffer command line input and dispatch handlers on Enter - Provide command registration API with per-command `user_data` - Provide machine-level `user_data` and optional per-tick hook for integration-specific logic - Encapsulate terminal raw mode lifecycle (enter/restore) inside the utility

Key Interfaces:

```

1  typedef struct cli_machine_data cli_machine_data;
2
3  typedef void (*cli_fsm_command_fn)(
4      rx_fsm_context *ctx,
5      cli_machine_data *cli,
6      const char *command,
7      void *command_user_data
8  );
9
10 typedef void (*cli_fsm_tick_fn)(
11     const rx_fsm_context *ctx,
12     cli_machine_data *cli,
13     void *user_data
14 );
15
16 void cli_fsm_data_init(cli_machine_data *data, void *user_data);
17 int cli_fsm_register_command(
18     cli_machine_data *data,
19     const char *name,
20     cli_fsm_command_fn handler,
21     void *command_user_data
22 );
23 void cli_fsm_create(rx_fsm_machine *machine, const char *name,
24     cli_machine_data *data);

```

Design Notes: - `main_cli.c` remains focused on runtime wiring and periodic ticking; command semantics live in command handlers - The CLI utility is reusable across examples because it has no domain-specific references - Command handlers can use both machine-level `cli->user_data` and per-command `command_user_data` depending on reuse needs

Data Models**Runtime Model**

```

1  interface RuntimeModel {
2      context: ContextModel
3      nodes: NodeModel[]
4      tickPhases: ['latch', 'evaluate', 'commit', 'deferred']
5  }
6
7  interface ContextModel {
8      inputs: unknown
9      latchedInputs: unknown
10     deferredActions: DeferredActionModel[]
11 }

```

```
12
13 interface DeferredActionModel {
14     fn: Function
15     user: unknown
16 }
```

FSM Model

```
1 interface FSMMachineModel {
2     name: string
3     state: number
4     nextState: number
5     transitions: FSMTransitionModel[]
6     user?: unknown
7 }
8
9 interface FSMTransitionModel {
10     fromState: number
11     toState: number
12     guard?: (ctx: unknown, user: unknown) => boolean
13     action?: (ctx: unknown, user: unknown) => void
14 }
```

PN Model

```
1 interface PNNetModel {
2     name: string
3     places: number[]
4     nextPlaces: number[]
5     transitions: PNTransitionModel[]
6     fireFlags: boolean[]
7     user?: unknown
8 }
9
10 interface PNTransitionModel {
11     consume: PNArcModel[]
12     produce: PNArcModel[]
13     guard?: (ctx: unknown, user: unknown) => boolean
14     action?: (ctx: unknown, user: unknown) => void
15 }
16
17 interface PNArcModel {
18     placeId: number
19     weight: number
20 }
```


Correctness Properties

A property is a behavior that should hold for all valid executions. Properties connect requirements with verifiable guarantees in unit, integration, and property-based tests.

Property 1: Phase Ordering Determinism

*For any tick execution, phase order should always be `latch` -> `evaluate` -> `commit` -> `deferred`. **Validates: Requirements 1.1, 1.2, 1.3, 1.4, 1.5***

Property 2: Snapshot Consistency Within Tick

*For any guard evaluation during one tick, observed inputs should come from each node's latched snapshot (taken in phase 1) and remain stable for that tick. **Validates: Requirements 2.1, 2.2, 2.3***

Property 3: Deferred Action Isolation

*For any action emitted during commit, execution should occur only after all nodes have completed commit. **Validates: Requirements 3.1, 3.2***

Property 4: Deferred Queue Reset

*For any completed tick, deferred queue length should be zero after running deferred actions. **Validates: Requirements 3.3***

Property 4b: Deferred Queue Overflow Determinism

*For any tick where deferred enqueue exceeds configured capacity, enqueue should return -1 and perform no dynamic allocation. **Validates: Requirements 3.4, 3.6, 14.6***

Property 5: Runtime Node Contract Safety

*For any node registered in the runtime, `evaluate` and `commit` should both be callable in every tick. **Validates: Requirements 4.1***

Property 6: Node Capacity Enforcement

*For any runtime initialized with capacity N , adding more than N nodes should fail with -1 . **Validates: Requirements 4.2, 4.3***

Property 6b: Init Capacity Bound Checks

*For any runtime/context initialization with capacities greater than configured maxima, initialization should fail with -1 . **Validates: Requirements 2.5, 4.5, 14.7***

Property 7: FSM First-Match Transition Rule

*For any FSM machine and state, transition selection should be the first declaration-order transition that matches state and guard. **Validates: Requirements 5.1, 5.2***

Property 8: FSM No-Match Stability

*For any FSM tick with no valid transition, machine state should remain unchanged. **Validates: Requirements 5.3***

Property 9: FSM Deferred Action Behavior

*For any matched FSM transition with action, the action should be queued for deferred execution, not run inline. **Validates: Requirements 5.5***

Property 10: FSM Input Snapshot Access

*For any FSM machine, guards should read from the latched input snapshot (captured in `latch_inputs_cb`) and not mutate it. **Validates: Requirements 6.1, 6.2, 6.3***

Property 11: PN Transition Enablement

*For any PN transition, firing should require valid arcs and sufficient consume tokens. **Validates: Requirements 7.2***

Property 12: PN Guard Enforcement

*For any enabled PN transition with guard, transition should fire only when guard is true. **Validates: Requirements 7.3***

Property 13: PN Commit Delta Correctness

*For any set of fireable PN transitions, place deltas should match arc consume/produce sums in commit. **Validates: Requirements 7.4***

Property 14: PN Arc Validation

*For any PN net init with invalid arc index or negative weight, init should fail with -1. **Validates: Requirements 8.1, 8.2***

Property 15: Lifecycle Idempotent Safety

*For any `*_free(NULL)` call, operation should be safe and not crash. **Validates: Requirements 9.5***

Property 16: Context Ownership Boundary

*For any runtime lifecycle, external input buffer should not be freed by runtime/context cleanup. **Validates: Requirements 2.4, 9.4***

Property 17: Example Executability

*For any provided C example entrypoint, example should run and complete without runtime errors in a valid toolchain. **Validates: Requirements 10.1, 10.2, 10.3***

Property 18: Host-Controlled Scheduling

*For any deployment context, tick frequency and loop ownership should remain controlled by host code, not by runtime internals. **Validates: Requirements 14.3***

Error Handling

Error Categories

- 1. API Misuse Errors (return-code based)** - Null pointers in required parameters - Capacity overflow in node registration - Invalid PN initialization data (arcs, buffers)
- 2. Allocation and Resource Errors** - Context buffer allocation failure - Deferred queue capacity exhaustion (enqueue rejection) - Net internal array allocation failure
- 3. Integration Errors** - Host passes invalid input lifetimes - Host loop omits input reset logic for edge-triggered signals

Error Handling Strategy

Explicit status codes: - Initialization and tick APIs return 0 on success, -1 on failure - Cleanup APIs are null-safe and best-effort - Partial init failures trigger cleanup of already allocated resources

Host strategy: explicit loop ownership - Host checks return codes per tick - Host applies retry/abort policy appropriate to runtime context (CLI/embedded)

Testing Strategy

Unit and Integration Tests

Unit Tests focus on: - Per-function semantics in runtime/fsm/pn layers - Edge cases (empty transitions, zero-capacity runtime, invalid arcs) - Correct deferred action behavior

Integration Tests focus on: - Host inputs mutation -> runtime tick -> state update -> deferred side effects - Multiple nodes sharing one context input struct - FSM guards reading shared latched inputs - C host example builds (gcc) for FSM and PN - ESP-IDF example compile-level validation

Property Tests focus on: - Tick-order invariants - PN token conservation and transition correctness across randomized nets - FSM deterministic transition ordering across randomized transition lists

Property-Based Testing Configuration

- C: randomized harness plus deterministic replay seeds
- Minimum 100 runs per property
- Each property references its design property number

- Seeded reproducibility for failure triage

Code Quality and Development Standards

- Compile with `-std=c11 -Wall -Wextra -Wpedantic`
- Preserve null-safety checks and explicit error returns
- Keep ownership boundaries explicit (runtime-owned vs host-owned memory)
- The tick path (`latch`, `evaluate`, `commit`, `run deferred actions`) SHALL perform no heap allocation

Non-Goals and Out-of-Scope

- No built-in networking or REST API layer
- No built-in persistence, serialization, or storage backends
- No built-in scheduler, thread pool, or realtime policy manager
- No automatic synchronization for concurrent access to one runtime instance

These concerns are intentionally delegated to host applications.

Implementation Plan: rxnet — C Implementation

Overview

This implementation plan converts the [rxnet](#) C design into discrete tasks for completing and validating the library. It follows an incremental approach: core runtime semantics first, model frontends second, integration examples third, and verification/quality automation last.

Tasks are marked as completed when they reflect the current repository state, and left pending where implementation, tests, or automation are still missing.

Tasks

- ☒ 1. Establish C repository structure
 - ☒ 1.1 Create C runtime/frontend directories and public headers
 - ★ Add `c/include/rxnet` and `c/src` layout for core runtime + FSM + PN
 - ★ *Requirements: 4.1, 9.1*
 - ☒ 1.2 Add top-level README docs
 - ★ Document phase model and basic execution usage
 - ★ *Requirements: 10.1*
- ☒ 2. Implement shared synchronous core runtime
 - ☒ 2.1 Implement context lifecycle and input latching
 - ★ Add `rx_context_init/free/latch_inputs`
 - ★ Implement `inputs + latched_inputs` model and ownership boundaries
 - ★ *Requirements: 1.1, 2.1, 2.2, 2.3, 2.4, 9.4*
 - ☒ 2.2 Implement deferred action queue
 - ★ Add enqueue/run APIs with fixed-capacity bounded behavior
 - ★ *Requirements: 3.1, 3.2, 3.3, 3.4, 3.5, 3.6, 14.6*
 - ☒ 2.3 Implement generic runtime node orchestration

- * Add runtime init/add/tick/free and phase ordering
- * *Requirements: 1.2, 1.3, 1.4, 1.5, 4.1, 4.2, 4.3, 4.4*

☒ 3. Implement FSM frontend

☒ 3.1 Implement machine model and transition semantics

- * First-match transition selection in declaration order
- * Commit state and enqueue deferred action
- * *Requirements: 5.1, 5.2, 5.3, 5.4, 5.5*

☒ 3.2 Implement FSM runtime wrapper

- * Runtime init/free, machine registration, tick wrapper
- * *Requirements: 9.1, 9.2*

☒ 3.3 Implement shared-input guard access

- * Read shared latched inputs directly from FSM guards/actions
- * *Requirements: 6.1, 6.2, 6.3*

☒ 4. Implement Petri Net frontend

☒ 4.1 Implement PN net model and transition execution

- * Evaluate fire flags from token availability + guards
- * Apply consume/produce deltas during commit
- * *Requirements: 7.1, 7.2, 7.3, 7.4, 7.5*

☒ 4.2 Implement PN validation and allocation safety

- * Validate arcs and reject invalid models
- * Handle allocation failures with cleanup
- * *Requirements: 8.1, 8.2, 8.3*

☒ 4.3 Implement PN runtime wrapper and lifecycle APIs

- * Runtime init/free, net add/tick, net free
- * *Requirements: 9.1, 9.2, 9.3, 9.5*

☒ 5. Provide runnable C examples and host integration patterns

☒ 5.1 Implement FSM and PN console examples

- * Add deterministic sample flows for state/token updates
- * *Requirements: 10.1, 14.3, 14.4*

☒ 5.2 Implement FSM 00-light host CLI and ESP-IDF variants

- * Shared input struct consumed directly by machine guards
- * *Requirements: 6.1, 10.2, 10.3, 14.3*

☒ 5.3 Implement FSM CLI examples 01-auto, 02-blink, 03-mix

- ★ Three-state blink, auto-off timer, and mixed runtime
- ★ *Requirements: 10.2, 10.3*
- ☒ 5.4 Implement PN CLI examples 01-light, 02-auto, 03-blink, 04-mix
 - ★ REQUEST places, AUTO_OFF_DUE signal place, TOGGLE_DUE signal place, dual runtime
 - ★ *Requirements: 10.2, 10.3*
- ☒ 6. Create baseline C specification documents
 - ☒ 6.1 Write C requirements specification
 - ★ Add `docs/specs/c/requirements.md` as source of truth for C behavior
 - ☒ 6.2 Write C design specification
 - ★ Add `docs/specs/c/design.md` with architecture, interfaces, and properties
- ☒ 7. Add C unit and integration tests
 - ☒ 7.1 Add runtime/context tests
 - ★ 26 tests: tick ordering, deferred queue FIFO + capacity limits, null-safety checks
 - ★ *Requirements: 1.1, 1.5, 3.3, 4.3, 4.4, 9.5*
 - ☒ 7.2 Add FSM semantic tests
 - ★ 21 tests: first-match ordering, false-guard skip, user-data guard reads, no-match stability, deferred action timing
 - ★ *Requirements: 5.2, 5.3, 6.2*
 - ☒ 7.3 Add PN semantic/validation tests
 - ★ 24 tests: arc validation, token availability, delta correctness, guard/action wiring, net lifecycle
 - ★ *Requirements: 7.2, 7.4, 8.1, 8.2, 9.3*
- ☒ 8. Add C/Python semantic parity test harness
 - ☒ 8.1 Define shared scenario corpus
 - ★ Three scenarios: `fsm_light` (toggle), `fsm_first_match`, `pn_light` (token toggle)
 - ★ Documented in `c/tests/parity_runner.c` and `python/tests/parity_runner.py`
 - ★ *Requirements: 1.5*
 - ☒ 8.2 Implement cross-language conformance runner (C side)
 - ★ `c/tests/parity_runner.c` emits 15 lines; `tests/parity/run_parity.sh` diffs C vs Python

- ★ Makefile `parity` target builds the runner; CI `parity` job runs the check
 - ★ *Requirements: 1.5*
- ☒ 9. Implement developer quality automation
 - ☒ 9.1 Add C static analysis and warning gates
 - ★ Build uses `-Wall -Wextra -Wpedantic`; `make test` is warning-clean
 - ★ `.github/workflows/ci.yml` `c-tests` job enforces this on every PR
 - ★ *Requirements: 9.4, 14.1*
 - ☒ 9.2 Add pre-commit and CI checks
 - ★ `.github/workflows/ci.yml`: `c-tests`, `python-tests`, `parity` jobs on push/PR
 - ★ `.pre-commit-config.yaml`: ruff + C test hook
 - ★ *Requirements: 10.1*
- ☒ 10. Add C library build/install workflow
 - ☒ 10.1 Define canonical build targets for library + examples
 - ★ Makefile builds all FSM + PN examples: `fsm_{light,auto,blink,mix}cli + pn{01,02,03,04}`
 - ★ *Requirements: 10.1, 14.3*
 - ☒ 10.2 Document versioning and compatibility rules
 - ★ Versioning policy: increment `version` in `docs/specs/c/requirements.md` preamble on any public API change; bump `RXNET_VERSION_*` macros in `config.h` for releases. PR checklist: requirements → design → tasks → code.
- ☒ 11. Final validation and readiness checkpoint
 - ☒ 11.1 Verify all correctness properties are covered by tests
 - ★ 71 C unit tests (`test_runtime`, `test_fsm`, `test_pn`) cover all design properties
 - ★ 15-line parity trace covers cross-language semantic equivalence (Property 1, 5, 11)
 - ☒ 11.2 Verify requirements-to-implementation traceability
 - ★ All requirements §1–§14 have corresponding implementation + `make test` coverage
 - ★ Parity harness covers §1.5 (cross-language conformance)
 - ☒ 11.3 Establish “requirements-first” change workflow
 - ★ PR rule encoded in `.github/workflows/ci.yml` and documented here: update `requirements.md` → `design.md` → `tasks.md` before behavioral code changes
- ☒ 12. Reflect reusable CLI FSM changes in specifications
 - ☒ 12.1 Update requirements for reusable CLI FSM contract

- ★ Capture command registry API, raw mode encapsulation, and user data behavior
 - ★ *Requirements: 10.3, 11.1, 11.2, 11.3, 11.4, 11.5*
- ☒ 12.2 Update design for reusable CLI FSM architecture and interfaces
 - ★ Document generic CLI FSM responsibilities and public C interfaces
 - ★ *Requirements: 11.1, 11.2, 11.3, 11.4, 11.5*
- ☒ 13. Enforce no dynamic allocation during tick path
 - ☒ 13.1 Update deferred queue implementation to fixed-capacity behavior
 - ★ Remove runtime `realloc` from deferred enqueue path and reject overflow deterministically
 - ★ *Requirements: 3.4, 3.6, 14.6*
 - ☒ 13.2 Update specs and traceability for bounded deferred queue
 - ★ Align requirement/design/task wording with fixed-capacity queue model
 - ★ *Requirements: 3.4, 3.6, 14.6*
 - ☒ 13.3 Build-check C examples after memory policy change
 - ★ Verify FSM and PN examples still compile cleanly
 - ★ *Requirements: 10.1, 9.1*
- ☒ 14. Configure fixed C runtime arrays via config header
 - ☒ 14.1 Add `rxnet/config.h` with runtime capacity macros
 - ★ Define compile-time maxima for latched inputs, deferred queue, and runtime nodes
 - ★ *Requirements: 14.7*
 - ☒ 14.2 Refactor C runtime context/node storage to fixed arrays
 - ★ Replace heap-backed runtime core buffers with arrays bounded by config macros
 - ★ *Requirements: 2.5, 3.4, 3.6, 4.5, 14.6, 14.7*
 - ☒ 14.3 Validate examples build after fixed-array migration
 - ★ Compile FSM/PN examples to confirm compatibility
 - ★ *Requirements: 10.1, 9.1*
- ☒ 15. Migrate C node model to embedding + add init/create API pair
 - ☒ 15.1 Refactor core runtime node API to typed embedded-node pointers
 - ★ Remove `void* self` adapter pattern and store `rx_node*` in runtime registry
 - ★ *Requirements: 4.1, 12.1, 12.4, 12.5*
 - ☒ 15.2 Update FSM/PN structs to embed `rx_node` base
 - ★ Make FSM and PN runtime registration use the embedded base member
 - ★ *Requirements: 12.2, 12.3, 12.4*

- ☒ 15.3 Add `_create/_destroy` convenience APIs alongside `_init`
 - ★ Implement create/destroy pairs for runtime/model entities while preserving init APIs
 - ★ *Requirements: 9.1, 9.6, 13.1, 13.2, 13.3, 13.4, 13.5*
- ☒ 15.4 Build-check C examples after API refactor
 - ★ Confirm all C examples still compile and run
 - ★ *Requirements: 10.1, 9.1*
- ☒ 16. Migrate canonical C examples to `_create/_destroy` APIs
 - ☒ 16.1 Update FSM console example to use create/destroy lifecycle
 - ★ Replace stack+init/free pattern with create/destroy pattern for runtime and machines
 - ★ *Requirements: 10.1, 13.1, 13.2, 13.3*
 - ☒ 16.2 Update PN console example to use create/destroy lifecycle
 - ★ Replace stack+init/free pattern with create/destroy pattern for runtime and net
 - ★ *Requirements: 10.1, 13.1, 13.2, 13.3*
 - ☒ 16.3 Build-check updated examples
 - ★ Compile and execute both examples after migration
 - ★ *Requirements: 10.1, 9.1*
- ☒ 17. Reorder C frontend runtime wrappers for base-runtime upcast
 - ☒ 17.1 Place `rx_runtime` as first member in FSM/PN runtime wrappers
 - ★ Enable safe upcast from frontend runtime pointer to base runtime pointer
 - ★ *Requirements: 12.6*
 - ☒ 17.2 Build-check C examples after wrapper layout change
 - ★ Ensure no behavioral regressions from struct field reordering
 - ★ *Requirements: 10.1, 9.1*
- ☒ 18. Eliminar código muerto y corregir brecha spec/implementación
 - ☒ 18.1 Eliminar `RXNET_MAX_INPUT_SIZE` de `config.h` y specs
 - ★ El macro está definido pero nunca referenciado en el código
 - ★ Eliminar de `config.h`, requirements §2 y design §Context
 - ☒ 18.2 Actualizar requirements §2 para reflejar el modelo real de inputs
 - ★ El contexto C solo tiene la cola de diferidos; los inputs son responsabilidad de cada nodo
 - ★ Eliminar referencias a `inputs/latched_inputs` en `rx_context`; documentar patrón real
 - ☒ 18.3 Actualizar design §Context y §API signatures para coincidir con implementación

- ★ Corregir firma `rx_context_init(ctx)` (no tiene parámetros de inputs)
- ★ Corregir firma `rx_fsm_machine_init` (incluye latch/dump callbacks)
- ★ Eliminar descripción de campos `inputs/latched_inputs` en `rx_context`
- ☒ 19. Unificar callbacks latch/dump en PN con convenio (`ctx`, `user`)
 - ☒ 19.1 Añadir campos `latch_inputs_fn` y `dump_outputs_fn` a `rx_pn_net`
 - ★ Tipo: `void (*) (rx_pn_context *ctx, void *user)`, igual que FSM
 - ★ La vtable interna sigue usando (`node`, `ctx`) pero delega a estos campos si están definidos
 - ☒ 19.2 Actualizar `rx_pn_net_init` para recibir y registrar ambas callbacks
 - ★ Añadir `latch_inputs_fn` y `dump_outputs_fn` como parámetros opcionales (NULL = noop)
 - ☒ 19.3 Migrar ejemplos PN a la nueva API de callbacks
 - ★ Eliminar el patrón de cast manual (`rx_pn_net *`) `node` en los ejemplos
 - ☒ 19.4 Actualizar tests y parity runner tras el cambio de API
 - ☒ 19.5 Actualizar requirements y design para documentar el nuevo convenio
- ☒ 20. Añadir documentación de patrones de concurrencia
 - ☒ 20.1 Añadir sección “Integration Patterns” a requirements §14
 - ★ Tres patrones: ejecutivo cíclico, threads con mutex, RTOS con notificación de tarea
 - ★ Describir qué proteger (escritura de inputs + llamada a tick)
 - ★ Describir cuándo no hace falta mutex (inputs word-sized atómicos en ARM Cortex-M)
 - ☒ 20.2 Añadir sección “Concurrency” a design §Host Integration Layer
 - ★ Código de ejemplo para cada patrón (pthread, FreeRTOS, ejecutivo cíclico)
 - ★ Diagrama de secuencia: writer → inputs → latch → tick → outputs
- ☒ 21. Añadir ejemplo de runtime mixto FSM + PN
 - ☒ 21.1 Implementar `c/examples/mixed/main_cli.c`
 - ★ Un runtime base `rx_runtime` con un `rx_fsm_machine` y un `rx_pn_net` registrados juntos
 - ★ Demuestra que ambos tipos de nodo comparten el mismo tick
 - ☒ 21.2 Añadir target `mixed` al Makefile
 - ☒ 21.3 Documentar el patrón en design §Host Integration Layer
- ☒ 22. Reescribir READMEs con guía real de integración
 - ☒ 22.1 Reescribir `c/README.md`

- ★ Corregir targets de make obsoletos ([pn_00_queue](#) no existe)
- ★ Añadir: qué es rxnet, cuándo usar FSM vs PN, patrón básico, los tres modelos de concurrencia
- ☒ 22.2 Reescribir [README.md](#) raíz
 - ★ Añadir getting-started, tabla FSM vs PN, enlace a ejemplos y specs

Notes

- Tasks marked with ★ are optional for early MVP but recommended for robustness.
- This file tracks implementation status against current repository state as of April 2026.
- Any behavior change should first update [docs/specs/c/requirements.md](#), then [docs/specs/c/design.md](#), then this task plan.